



AhamX Bodhi

Where I Becomes Infinite

Image Embeddings

Module 4.1: Vision-Language Models

Prof. Sashikumaar Ganesan | IISc Bengaluru | 2026



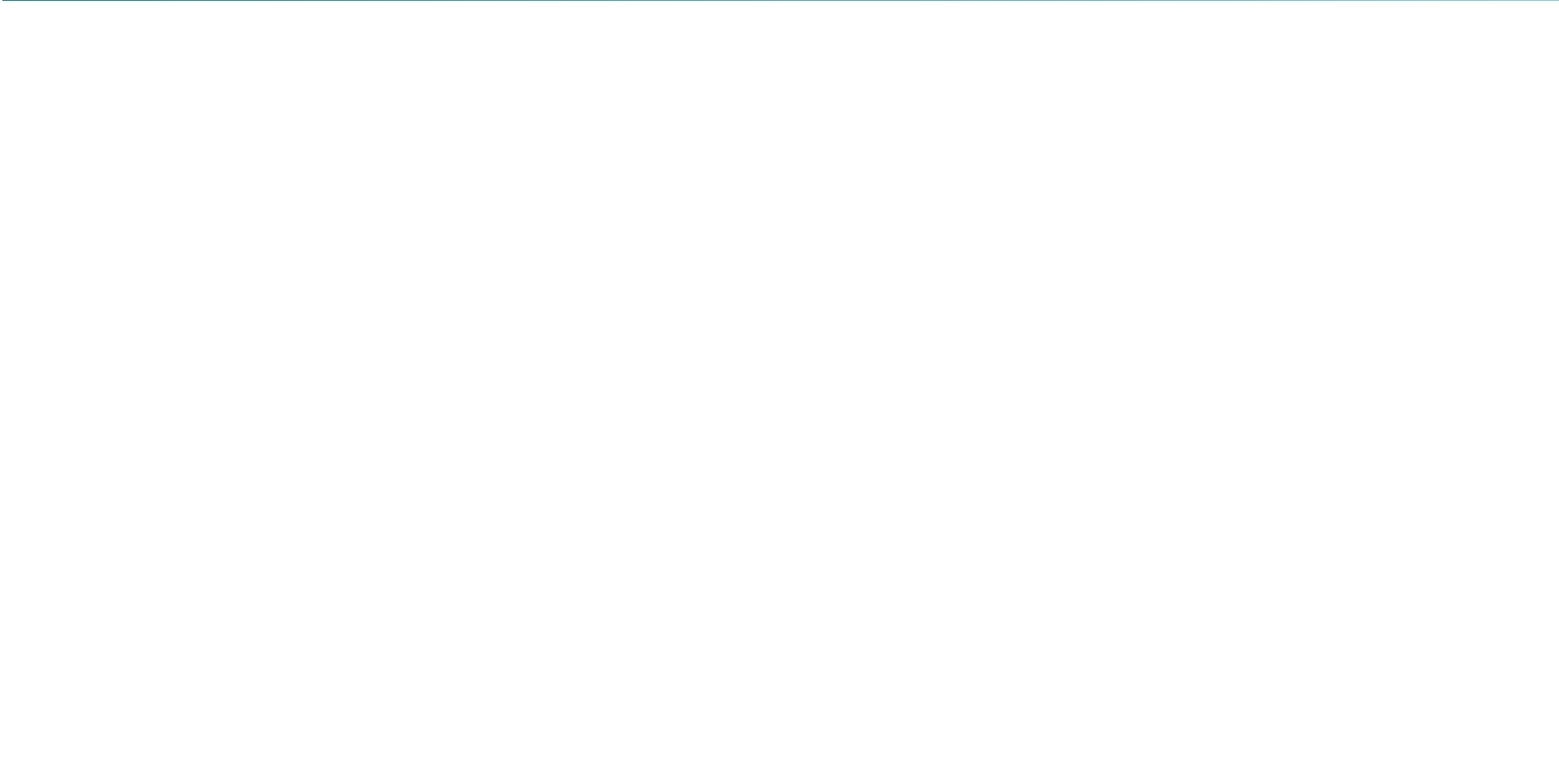
Session Overview

What You Will Learn

- What image embeddings are and why they exist
- Patch embeddings vs. global embeddings
- How image embedding spaces are learned
- Alignment with text embedding spaces
- Production use cases for AI engineers

Concept at a Glance

- **Duration:** 9 minutes
- **Bloom's Level:** B3 – Apply
- **Difficulty:** L2 – Intermediate
- **Module:** 4.1 Vision-Language Models
- **Prerequisite:** ViT architecture (TH-000002)





From Pixels to Vectors

The Core Idea

- **Raw image:** A 3D tensor of pixel values – e.g., $224 \times 224 \times 3 = 150,528$ numbers. Meaningless for similarity computation.
- **Image embedding:** A dense, fixed-length vector (e.g., 512-dim or 1024-dim) in a learned metric space where *semantic similarity corresponds to geometric proximity*
- **Key property:** Two images of cats are closer in embedding space than a cat and a car, regardless of pixel-level differences
- **Learned, not engineered:** The embedding space is learned from data – not hand-crafted features like SIFT or HOG
- **Two types:** Global embedding (one vector per image – classification, retrieval) and patch embeddings (one vector per patch – VQA, grounding)

Image embeddings are to computer vision what word embeddings are to NLP – the foundational representation enabling all downstream reasoning.



How ViT Produces Patch Embeddings

dim

Step-by-Step: Image to Embedding

- **Step 1 – Patchify:** Divide image into $P \times P$ non-overlapping patches. For 224×224 with $P = 16$: 196 patches of shape $16 \times 16 \times 3 = 768$ values
- **Step 2 – Linear projection:** Multiply each flattened patch by a learned weight matrix $E \in \mathbb{R}^{768 \times D}$ – this is the “patch embedding layer”
- **Step 3 – Add positional encoding:** Add learned position embeddings $E_{pos} \in \mathbb{R}^{N \times D}$ to preserve spatial order
- **Step 4 – Self-attention:** L Transformer encoder layers refine patch embeddings through mutual attention – each patch attends to every other patch
- **Step 5 – Output:** Final patch embeddings are *contextual* – each patch representation encodes global image context

Unlike CNN features where each spatial cell only “sees” its local receptive field, ViT patch embeddings are globally aware from the first encoder layer.



Global Embedding vs. Patch Embeddings

Property	Global Embedding (CLS)	Patch Embeddings (All)
Shape	$[1 \times D]$	$[N \times D]$
Encodes	Whole-image semantics	Spatial / local details
Used for	Classification, retrieval, RAG	VQA, grounding, segmentation
Memory	Minimal	$N \times$ more (e.g., $196 \times$)
Example	CLIP image embedding	LLaVA visual tokens

For VLMs, the patch embeddings (not just CLS) are passed to the language model – this is why “image understanding” requires feeding potentially 256–1024 visual tokens per image into the LLM context.

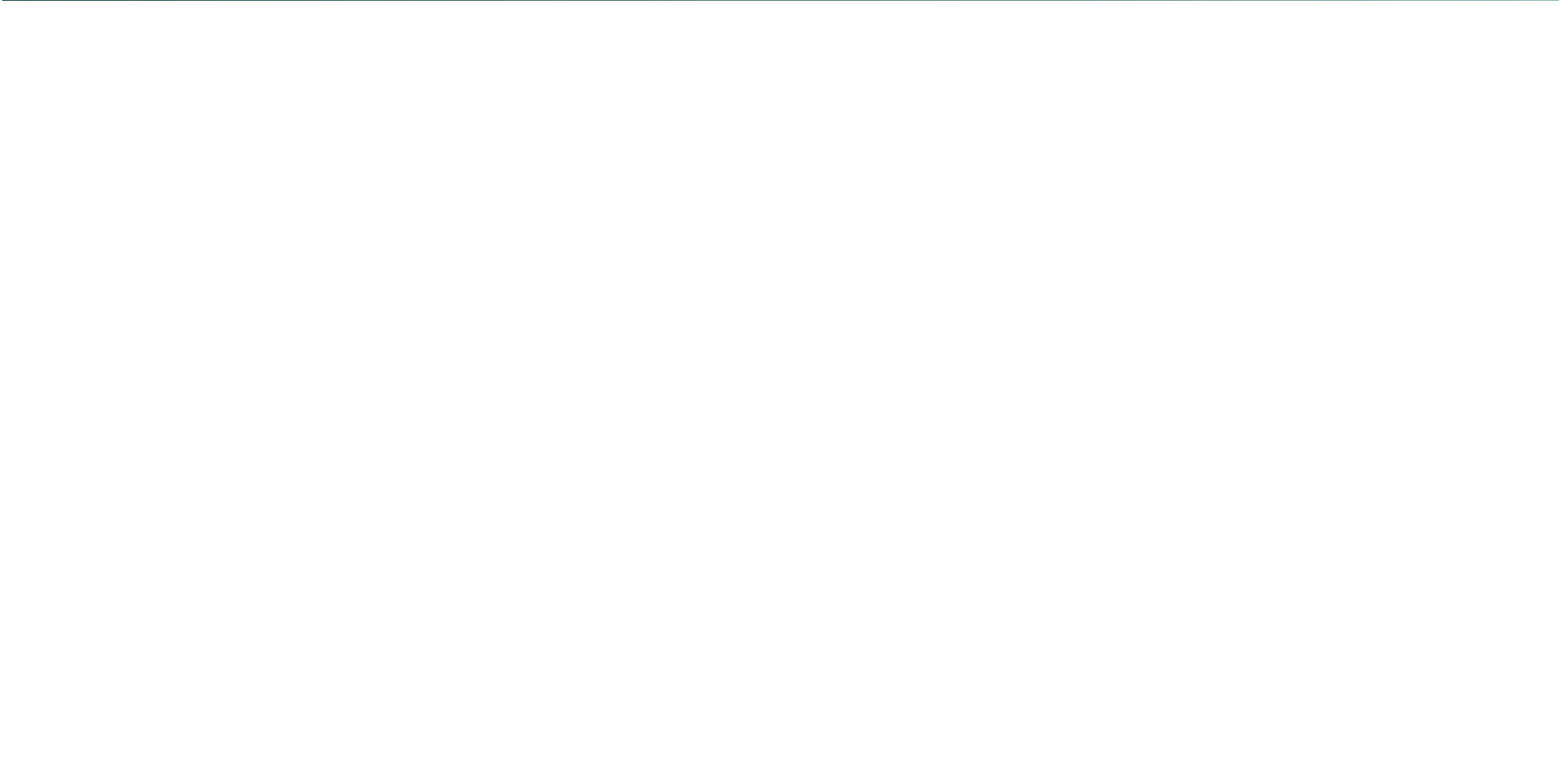


Properties of a Well-Learned Embedding Space

What Good Embeddings Enable

- **Semantic clustering:** Dogs cluster together, cars cluster together – even with different breeds, colors, or backgrounds
- **Analogical reasoning:** $\vec{dog} - \vec{puppy} \approx \vec{adult_dog}$ – arithmetic in embedding space is meaningful
- **Cross-modal alignment (CLIP):** “a photo of a dog” $\approx$ dog_photo – text and image embeddings of the same concept are nearby
- **Transferability:** Features learned on ImageNet transfer to medical imaging, satellite images, product photos – learned priors generalize
- **Measurable similarity:** Cosine similarity and dot product become reliable semantic proximity metrics

The quality of image embeddings determines the quality of every downstream task – retrieval precision, VQA accuracy, and generation coherence all depend on the embedding space.





How Image Embedding Spaces Are Learned

Training Objectives

- **Supervised (ImageNet):** Cross-entropy loss on class labels. Good for object recognition, limited generalization to open-vocabulary
- **Contrastive (CLIP, SigLIP):** Pull matching image-text pairs together, push non-matching pairs apart. Learns rich semantic features aligned with language
- **Self-supervised (DINO, MAE):** No labels – predict masked patches (MAE) or use self-distillation (DINO). State-of-the-art for general-purpose features
- **Masked image modeling (BEiT, MAE):** Mask 75% of patches, reconstruct pixel values or discrete tokens. Learns dense spatial features critical for segmentation

Self-supervised pre-training (MAE + fine-tuning) has largely replaced supervised pre-training as the dominant paradigm for foundation vision models.



CLIP vs. DINOv2: Two Embedding Philosophies

Contrastive Language-Image (CLIP) vs. Self-Supervised (DINOv2)

- **CLIP space:** Aligned with natural language – visual concepts map to text descriptions. Ideal for zero-shot classification and cross-modal retrieval
- **DINOv2 space:** Purely visual – fine-grained texture and shape discrimination not tied to language. Ideal for visual similarity and dense prediction
- **Practical difference:** DINOv2 better distinguishes visually similar objects of different types; CLIP better handles open-vocabulary queries from text

Which to Use?

- Text query → image results: CLIP / SigLIP
- Image query → similar images: DINOv2
- VLM visual backbone: SigLIP-SO (best text-image alignment for generation tasks)





GenAI Application: Image Similarity Search

Building a Visual Product Search System

- **Offline (indexing):** Extract DINOv2 ViT-B/14 embeddings for all catalog images; store 768-dim vectors in Qdrant or FAISS with product metadata
- **Online (query):** User uploads photo → encode with same DINOv2 model → ANN search (HNSW index) → return top-k similar products in <50ms
- **Scale:** Zalando, Pinterest, and IKEA use ViT-based embeddings for visual search over millions of products
- **Hybrid:** Combine visual embedding similarity with text embedding similarity (CLIP) for “find a red sofa like this but smaller” – multi-modal filtering

FAISS flat index for <1M vectors; HNSW or IVF-PQ for >10M vectors. At 768-dim fp32, 10M images = 29GB – use fp16 or int8 quantization for cost reduction.



GenAI Application: Image Embeddings in Multimodal RAG

Retrieving Images for Vision-Language Generation

- **Problem:** Technical documentation contains diagrams whose content is not captured by text-only OCR
- **Solution:** Index document page images as CLIP embeddings alongside text chunks; at query time retrieve both relevant text and images
- **ColPali (2024):** Late-interaction model that produces per-patch embeddings from document page images – retrieves at the patch level, not just globally. Achieves state-of-the-art on DocVQA retrieval
- **System:** Retrieved image(s) + text question passed to GPT-4V or LLaVA for grounded answer generation

Key Insight

ColPali eliminates the need for OCR preprocessing – the vision encoder directly produces retrievable representations from rendered page images.



GenAI Application: Medical Image Embedding

Domain-Specific Embedding Fine-Tuning

- **Problem:** General-purpose CLIP/DINOv2 embeddings are not optimal for medical images – trained on natural photos, not X-rays or pathology slides
- **Solution:** Domain-adaptive fine-tuning using contrastive learning on medical image-report pairs (BioViL-T, CheXzero, CONCH for pathology)
- **BioViL-T (Microsoft, 2023):** ViT fine-tuned on chest X-ray–radiology report pairs; enables zero-shot pneumonia detection and report-to-image retrieval
- **UNI (2024):** General-purpose pathology foundation model trained on 100k+ whole-slide images; ViT-L backbone with 1B training patches
- **Engineer's task:** Replace CLIP ViT with domain-specific ViT in the embedding pipeline; rest of the stack (vector DB, retrieval) is identical

Domain-specific embedding models consistently outperform general-purpose models on specialized datasets – fine-tuning on ~50k domain images is often sufficient.



Engineering: Embedding Dimensionality and Storage

Storage and Compute Trade-offs

- **Common dimensions:** 512 (CLIP ViT-B), 768 (DINOv2 ViT-B), 1024 (CLIP ViT-L, SigLIP), 1280 (ViT-H)
- **Matryoshka Representation Learning (MRL):** Train embeddings that are accurate at any sub-dimension – deploy 256-dim for speed, 1024-dim for precision. OpenAI's text-embedding-3 uses MRL
- **Quantization:** fp32 $\rightarrow$ int8 reduces storage 4 $\times$ with <1% recall loss using product quantization (FAISS PQ)
- **Binary embeddings:** 1-bit vectors enable 32 $\times$ compression – viable for initial filtering before full-precision reranking

Practical Rule of Thumb

Start with 768-dim DINOv2 fp16 for general visual search. Only increase to 1024+ if recall metrics justify the 33% storage and latency cost.



Key Takeaways

What You Should Remember

- Image embeddings are dense vectors in a learned semantic space – geometric distance encodes visual and semantic similarity
- Two types: global (CLS token) for retrieval and classification; patch embeddings for spatial reasoning in VLMs
- Training objectives: supervised (cross-entropy), contrastive (CLIP/SigLIP), self-supervised (DINO/MAE) – each optimizes for different downstream tasks
- SigLIP for VLM backbones; DINOv2 for visual similarity; CLIP for cross-modal retrieval
- Storage engineering: use fp16, int8 quantization, or MRL sub-dimensions to scale to billions of images cost-effectively

1. Path emb matrix

2. Positional embedding

3. attention weights (Q, K, V)

4. MLP (FFN layer)

5. Classifier

$\Rightarrow$ learnable parameters in Viz.

AhamX Bodhi

Where I Becomes Infinite